

AUTHENTICATION AND ACCESS CONTROL

1. Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

```
<!DOCTYPE html>

<html>

<head>

  <title>Zomato Admin Login</title>

</head>

<body>

  <h2>Zomato Admin Login</h2>

  <input type="text" id="username" placeholder="Username"><br><br>
  <input type="password" id="password" placeholder="Password"><br><br>

  <button onclick="login()">Login</button>

  <h3 id="result"></h3>

  <script>
    function login() {
      const user = document.getElementById("username").value;
      const pass = document.getElementById("password").value;
```

```
if (user === "admin" && pass === "admin123") {  
    document.getElementById("result").innerHTML = "Access Granted";  
} else {  
    document.getElementById("result").innerHTML = "Access Denied";  
}  
}  
</script>  
</body>  
</html>
```

Zomato Admin Login

Access Granted

**2. Write a Python function `check_access(user_role, resource)` that returns `True` if the `user_role` is allowed to access the resource, otherwise `False`. Test it with roles like `'customer'`, `'restaurant_owner'`, and resources like `'menu_edit'`, `'order_history'`, `'admin_dashboard'`.
Hint: Use a dictionary to map roles to allowed resources.**

```
permissions = {  
    "customer": ["order_history"],  
    "restaurant_owner": ["menu_edit", "order_history"],  
    "admin": ["menu_edit", "order_history", "admin_dashboard"]  
}
```

```
def check_access(user_role, resource):
```

```
    return resource in permissions.get(user_role, [])
```

```
print(check_access("customer", "menu_edit"))    # False  
print(check_access("customer", "order_history")) # True  
print(check_access("restaurant_owner", "menu_edit")) # True  
print(check_access("admin", "admin_dashboard"))  # True
```

```
Run started  
Initializing environment  
Installing packages  
Running code  
False  
True  
True  
True  
Run completed in 2260.099999997765ms
```

3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

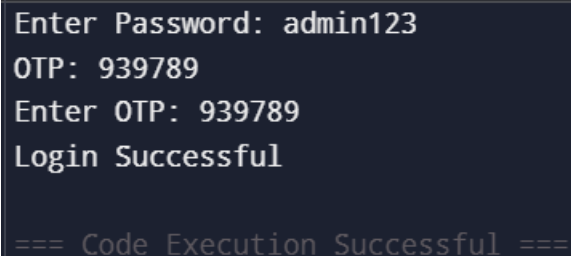
```
import random

password = input("Enter Password: ")

if password == "admin123":
    otp = random.randint(100000, 999999)
    print("OTP:", otp)

    user_otp = input("Enter OTP: ")

    if user_otp == str(otp):
        print("Login Successful")
    else:
        print("Invalid OTP")
else:
    print("Wrong Password")
```



```
Enter Password: admin123
OTP: 939789
Enter OTP: 939789
Login Successful

=== Code Execution Successful ===
```

4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7

**days.

Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.**

```
from datetime import datetime
```

```
today = datetime.strptime("2024-06-10", "%Y-%m-%d")
```

```
users = [  
    {"username": "Amit", "isPremium": True, "lastLogin": "2024-06-08"},  
    {"username": "Rahul", "isPremium": False, "lastLogin": "2024-06-09"},  
    {"username": "Priya", "isPremium": True, "lastLogin": "2024-05-25"}  
]
```

```
def get_active_premium(users):  
    result = []  
    for user in users:  
        login_date = datetime.strptime(user["lastLogin"], "%Y-%m-%d")  
        if user["isPremium"] and (today - login_date).days <= 7:  
            result.append(user)  
    return result
```

```
print(get_active_premium(users))
```

```
[{'username': 'Amit', 'isPremium': True, 'lastLogin': '2024-06-08'}]
```

5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip

Mistake	Fix/Prevention
1. Weak passwords	Use strong passwords and password policies.
2. No role-based access control	Give users only the permissions they need.
3. Missing authentication checks	Verify the user's identity before allowing access.
4. Exposing admin pages to everyone	Restrict admin pages to authorized users only.
5. No two-factor authentication	Enable OTP or multi-factor authentication for better security.